Programlama Dilleri

Dr. Öğr. Üyesi Hayri Volkan Agun

Bilgisayar Mühendisliği Bölümü

Bursa Teknik Üniversitesi

Fonksiyon Parametreler

- Bazı algoritmalar, başka fonksiyonları parametre olarak almalıdır. Bu, yüksek seviyeli soyutlama sağlar.

```python
def uygula(f, x):
    return f(x)

def kare(x):
    return x*x

uygula(kare, 5)
```

- Bir fonksiyonun altında kalan alan, farklı noktalardan örnek alınarak hesaplanır.

```python
def integrate(f, a, b, n):
    dx = (b-a)/n
    return sum(f(a+i*dx) for i in range(n))*dx
```

Fonksiyon Parametreler

- Aynı integrasyon fonksiyonu farklı fonksiyonlar için kullanılabilir. Bu yazılım mühendisliğinde kritik bir ilkedir.

```python
integrate(lambda x: x*x, 0, 1, 100)
integrate(lambda x: x**3, 0, 1, 100)
```

- Bir fonksiyonun altında kalan alan, farklı noktalardan örnek alınarak hesaplanır.

```python
def integrate(f, a, b, n):
    dx = (b-a)/n
    return sum(f(a+i*dx) for i in range(n))*dx
```

Fonksiyon Parametreler

- Fonksiyonların (alt programların) parametre olarak başka fonksiyonlara gönderilmesi, yazılım mühendisliğinde yüksek derecede soyutlama ve yeniden kullanılabilirlik sağlar.
- Ancak bu yaklaşım, yüzeyde basit görünmesine rağmen dil tasarımı açısından iki temel ve karmaşık problemi beraberinde getirir.

```
</> C++

int kare(int x) { return x * x; }
float kup(float x) { return x * x * x; }

int (*fptr)(int);
fptr = kup;  // HATA (tip uyuşmazlığı)
```

- **Tip Denetimi (Type Checking) Problemi**
  - Fonksiyon parametre olarak gönderildiğinde, çağrılan fonksiyonun:
  - Kaç parametre aldığı
  - Parametre tiplerinin ne olduğu
  - Geri dönüş tipinin ne olduğu
- gibi özelliklerin kesin olarak bilinmesi gerekir.
- Aksi halde:
  - Derleme hataları oluşabilir
  - Yanlış fonksiyon çağrıları gerçekleşebilir
  - Runtime hataları ortaya çıkabilir
  - Bu nedenle birçok dil, fonksiyon pointer veya delegate gibi yapılarda **fonksiyon protokolünü (signature)** tipin bir parçası olarak tutar.

Fonksiyon Parametreler

```
let x = 10;

function yazdir() {
  console.log(x);
}

function degistir(f) {
  let x = 20;
  f();
}

degistir(yazdir);
```

- yazdir() çağrıldığında hangi x kullanılır?
  - 10 mu?
  - 20 mi?
- Cevap kullanılan **binding modeline bağlıdır**

- **Referans Ortamı (Referencing Environment) Problemi**
  - Fonksiyon başka bir fonksiyona parametre olarak gönderildiğinde kritik soru şudur:
  - **Bu fonksiyon çalıştırıldığında hangi değişkenlere erişebilecek?**
  - Bu durum özellikle:
    - İç içe (nested) fonksiyonlarda
    - Statik/dinamik scope farklarında
    - çok ciddi anlam karmaşalarına yol açar.

Atama

- Fonksiyon adı adres olarak kullanılır.

```
C++

int add(int a, int b) { return a+b; }
fptr = add;
```

- Pointer üzerinden fonksiyon çağrılabilir.

```
C++

fptr(2,3);
```

Binding Türleri

- Fonksiyonların parametre olarak gönderilmesi durumunda en kritik problem, fonksiyon çalıştırıldığında hangi değişken ortamının kullanılacağıdır. Bu problem, programlama dillerinde bağlama (binding) kavramı ile çözülür.

- Bu üç yaklaşım:
  - Shallow Binding
  - Deep Binding
  - Ad Hoc Binding
- Bu yaklaşımlar, fonksiyonun çalıştığı anda hangi scope'un aktif olacağını belirler.

```
</> pseudo

fonksiyon f = gönderilen fonksiyon
çağır(f)
```

Binding Türleri

- Fonksiyonların parametre olarak gönderilmesi durumunda en kritik problem, fonksiyon çalıştırıldığında hangi değişken ortamının kullanılacağıdır. Bu problem, programlama dillerinde bağlama (binding) kavramı ile çözülür.
- Binding, özellikle **lexical scope vs dynamic scope** tartışmasının merkezindedir.
- Bu örnek, binding türlerinin etkisini açıkça gösteren klasik bir senaryodur. İç içe fonksiyonlar ve farklı scope'lar kullanılarak, aynı fonksiyonun farklı sonuçlar üretmesi sağlanır.

JavaScript

```javascript
function sub1() {
 var x = 1;
 function sub2() { console.log(x); }
 function sub3() {
   var x = 3;
   sub4(sub2);
 }
 function sub4(f) {
   var x = 4;
   f();
 }
 sub3();
}
```

Binding Türleri

- sub2 → x kullanıyor. x üç farklı yerde tanımlı:
  - sub1 → 1
  - sub3 → 3
  - sub4 → 4
- 👉 Hangi x kullanılacak?

JavaScript

```javascript
function sub1() {
 var x = 1;
 function sub2() { console.log(x); }
 function sub3() {
   var x = 3;
   sub4(sub2);
 }
 function sub4(f) {
   var x = 4;
   f();
 }
 sub3();
}
```

Shallow Binding

- Shallow binding yaklaşımında, fonksiyonun çalıştırıldığı yerin ortamı kullanılır.
- Fonksiyon nerede çağrıldıysa o scope kullanılır.
- x = 20 olur.
  - Dinamik scope'a benzer.
  - Öngörülmesi zor sonuçlar üretir.
  - Modern dillerde nadiren kullanılır.

JavaScript

```javascript
function f() { console.log(x); }

function g(func) {
  let x = 20;
  func();
}

let x = 10;
g(f);
```

Deep Binding

- Deep binding, fonksiyonun tanımlandığı ortamı kullanır.
- Bu yaklaşım statik scope ile uyumludur.
- Modern programlama dillerinin çoğunda kullanılır.
- x = 10 olur.
  - En öngörülebilir yöntem.
  - Derleyici tarafından analiz edilebilir.
  - Closure yapısının temelidir

JavaScript

```javascript
let x = 10;

function f() { console.log(x); }

function g(func) {
  let x = 20;
  func();
}

g(f);
```

Ad Hoc Binding

- Ad hoc binding, fonksiyonun parametre olarak gönderildiği ortamı kullanır.
- Burada x = 30 olur.
  - Pratikte kullanılmaz.
  - Mantıksal bağlantı zayıftır.

JavaScript

```javascript
function f() { console.log(x); }

function g(func) {
  let x = 30;
  h(func);
}

function h(func) {
  let x = 40;
  func();
}
```

Kıyaslama

```javascript
let x = 5;

function f() { console.log(x); }

function g(func) {
  let x = 6;
  func();
}
```

- Shallow binding
  - x = 6
- Deep binging
  - x = 5

Çağrılma

- Fonksiyonun çağrılacağı yer çalışma zamanında belirlenir.
- Asenkron programlamanın temelidir.

C++

```cpp
void (*fp)();
fp = func;
fp();
```

Python

```python
def f(): print("hello")
g = f
g()
```

- Bir fonksiyon, başka bir fonksiyon tamamlandığında çağrılır.

JavaScript

```javascript
setTimeout(() => console.log("Bitti"), 1000);
```

Python

```python
def islem(f):
    print("Başladı")
    f()

islem(lambda: print("Tamamlandı"))
```

CSharp Delegate

- Delegate, fonksiyon pointer'ın nesneleştirilmiş halidir.

```csharp
public delegate int Change(int x);
```

```csharp
Change c = fun;
c(10);
```

- Bir delegate birden fazla fonksiyonu tutabilir.

```csharp
c += fun2;
```

```csharp
c += fun3;
c(5); // hepsi çalışır
```

Overloading

- Aynı isimli farklı fonksiyonlar.

```
Java

int f(int x)
float f(float x)
```

```
C++

int sum(int a, int b);
double sum(double a, double b);
```

- Kod okunabilirliği artar.
- Tip dönüşümleri belirsizlik yaratır.

```
C++

void f(int x);
void f(double x);

f(5); // hangisi?
```

```
C++

f(5.0f);
```

Varsayılan Parametre

```cpp
void f(int x=0);
void f();

f(); // hata
```

```cpp
void g(int x, int y=5);
g(3);
```

Genel Fonksiyon (Generic)

- Generic (jenerik) fonksiyonlar, aynı algoritmanın farklı veri tipleri üzerinde tekrar yazılmadan kullanılmasını sağlar.
- Bu yaklaşım **parametrik polimorfizm** olarak adlandırılır.
  Yani:
- Fonksiyonun davranışı sabittir
- Ancak veri tipi parametre olarak belirlenir
- Bu sayede:
  - Kod tekrarının önüne geçilir
  - Yazılım bakımı kolaylaşır
  - Tip güvenliği korunur

```cpp
template<class T>
T max(T a, T b) {
    return a > b ? a : b;
}
```

T burada bir tip parametresidir.
Derleyici bu fonksiyonu kullanılan tipe göre üretir.

```cpp
int a = max(3, 5);        // int versiyonu
double b = max(2.5, 4.1); // double versiyonu
```

Derleyici bu fonksiyon için iki ayrı versiyon oluşturur.

Genel Fonksiyon (Generic)

- Generic yapılar:
  - Compile-time polymorphism sağlar.
  - Runtime maliyet oluşturmaz.
  - Ancak kod çoğaltımı (code bloat) oluşturabilir.

```cpp
template<class T>
void sort(T arr[], int n) {
    for(int i=0;i<n;i++)
        for(int j=i+1;j<n;j++)
            if(arr[i] > arr[j])
                swap(arr[i], arr[j]);
}
```

Aynı sıralama algoritması:

int
float
char

gibi farklı tiplerde kullanılabilir.

Genel Fonksiyon (Generic)

- C dilinde generic yapıların yerine makrolar kullanılmıştır. Ancak makrolar metinsel genişleme yaptığı için ciddi hatalara neden olabilir.

```cpp
#define max(a,b) ((a)>(b)?(a):(b))
```

```cpp
int x = 5, y = 3;
max(x++, y);
```

Genişletilmiş hali

```cpp
((x++) > (y) ? (x++) : (y))
```

Eğer x > y ise x iki kez artırılır. Beklenen davranış bozulur.

Genel Fonksiyon – Java (Generic)

- Java’da generic yöntemler, C++’tan farklı olarak runtime’da tek kopya üzerinden çalışır.
- Primitive tipleri desteklemez (int yerine Integer).
- Tek kod kullanır ve daha az bellek gerektirir. Ancak runtime maliyeti vardır.

```java
public static <T> T getFirst(T[] arr) {
    return arr[0];
}
```

```java
String[] names = {"Ali", "Veli"};
System.out.println(getFirst(names));
```

```java
public static <T extends Comparable<T>> T max(T a, T b) {
    return a.compareTo(b) > 0 ? a : b;
}
```

T → Comparable olmak zorundadır. Bu sayede karşılaştırma yapılabilir.

Closure

- Closure, bir fonksiyon ile birlikte onun tanımlandığı ortamın (lexical environment) birleşimidir.
- Bu şu anlama gelir:
  - Fonksiyon, dış değişkenlere erişebilir.
  - Bu değişkenler fonksiyon yaşam süresi boyunca korunur.

JavaScript

```javascript
function makeAdder(x) {
  return function(y) {
    return x + y;
  }
}
```

JavaScript

```javascript
let add10 = makeAdder(10);
add10(5); // 15
```

x = 10 saklanır. Fonksiyon çağrılsa bile kaybolmaz.

Closure - Python

```python
def make_adder(x):
    def adder(y):
        return x + y
    return adder

add5 = make_adder(5)
print(add5(3))  # 8
```

- Closure için:
  - Static scope gerekir.
  - Nested function gerekir.
  - Heap allocation gerekebilir.
- Closure’ların en kritik özelliği, değişkenlerin yaşam süresini uzatmalarıdır.

Örnek - Closure

JavaScript

```javascript
function counter() {
  let count = 0;
  return function() {
    count++;
    return count;
  }
}
```

JavaScript

```javascript
let c = counter();
c(); //1
c(); //2
```

- count değişkeni silinmez.
- Heap’te tutulur. Aşağıdaki örnekte closure kullanılmıştır. Bu çıktıda i değeri heap’te hep 3 olacaktır.

JavaScript

```javascript
for(var i=0;i<3;i++){
  setTimeout(()=>console.log(i),1000);
}
```

Coroutine

- Coroutine, klasik fonksiyonlardan farklı olarak:Tek giriş yerine çoklu giriş noktası içerir.
- Durumunu korur.
- Resume mekanizması ile çalışır.

```pseudo
co1:
  print("A")
  resume co2
  print("B")
```

Coroutine

- Coroutine, klasik fonksiyonlardan farklı olarak:Tek giriş yerine çoklu giriş noktası içerir.
- Durumunu korur.
- Resume mekanizması ile çalışır.

```pseudo
co1:
  print("A")
  resume co2
  print("B")
```

```python
def gen():
    yield 1
    yield 2

g = gen()
print(next(g)) #1
print(next(g)) #2
```

Coroutine’ler:Thread’e alternatif hafif yapı sunar. Cooperative multitasking sağlar. Modern dillerde (Python, Lua) yaygındır.